

mnist

```
1 import numpy as np
2 import tensorflow as tf
3 import matplotlib.pyplot as plt
4 from tensorflow.keras import layers, models
5 (x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
6 x_train = x_train.astype("float32") / 255.0
7 x_test = x_test.astype("float32") / 255.0
```

```
1 noise = 0.5
2 x_train_noisy = x_train + noise * np.random.normal(size=x_train.shape)
3 x_test_noisy = x_test + noise * np.random.normal(size=x_test.shape)
4 x_train_noisy = np.clip(x_train_noisy, 0., 1.)
5 x_test_noisy = np.clip(x_test_noisy, 0., 1.)
6
7 x_train = np.expand_dims(x_train, -1)
8 x_test = np.expand_dims(x_test, -1)
9 x_train_noisy = np.expand_dims(x_train_noisy, -1)
10 x_test_noisy = np.expand_dims(x_test_noisy, -1)
```

```
1 input_img = layers.Input(shape=(28, 28, 1))
2 x = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(input_img)
3 x = layers.MaxPooling2D((2, 2), padding="same")(x)
4 x = layers.Conv2D(16, (3, 3), activation="relu", padding="same")(x)
5 encoder = layers.MaxPooling2D((2, 2), padding="same")(x)
```

```
1
2 x = encoder
3
4 x = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(x)
5 x = layers.UpSampling2D((2, 2))(x)
6
7 x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
8 x = layers.UpSampling2D((2, 2))(x)
9
10 decoder = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
```

```
1 autoencoder = models.Model(input_img, decoder)
2 autoencoder.compile(optimizer="nadam", loss="binary_crossentropy")
3 autoencoder.fit(x_train_noisy, x_train, epochs=18, batch_size=128, shuffle=True, validation_data=(x_test_noisy, x_test))
```

```
Epoch 1/18
391/391 ————— 10s 18ms/step - loss: 0.5950 - val_loss: 0.5956
Epoch 2/18
391/391 ————— 3s 9ms/step - loss: 0.5948 - val_loss: 0.5958
Epoch 3/18
391/391 ————— 3s 8ms/step - loss: 0.5948 - val_loss: 0.5962
Epoch 4/18
391/391 ————— 4s 9ms/step - loss: 0.5948 - val_loss: 0.5971
Epoch 5/18
391/391 ————— 3s 8ms/step - loss: 0.5948 - val_loss: 0.5962
Epoch 6/18
391/391 ————— 3s 8ms/step - loss: 0.5947 - val_loss: 0.5960
Epoch 7/18
391/391 ————— 3s 9ms/step - loss: 0.5947 - val_loss: 0.5958
Epoch 8/18
391/391 ————— 4s 9ms/step - loss: 0.5947 - val_loss: 0.5954
Epoch 9/18
391/391 ————— 3s 8ms/step - loss: 0.5947 - val_loss: 0.5969
Epoch 10/18
391/391 ————— 3s 8ms/step - loss: 0.5946 - val_loss: 0.5955
Epoch 11/18
391/391 ————— 4s 10ms/step - loss: 0.5946 - val_loss: 0.5954
Epoch 12/18
391/391 ————— 3s 8ms/step - loss: 0.5946 - val_loss: 0.5965
Epoch 13/18
391/391 ————— 3s 8ms/step - loss: 0.5946 - val_loss: 0.5966
Epoch 14/18
391/391 ————— 3s 9ms/step - loss: 0.5945 - val_loss: 0.5956
Epoch 15/18
178/391 ————— 1s 9ms/step - loss: 0.5952
```

binary accuracy of mnist

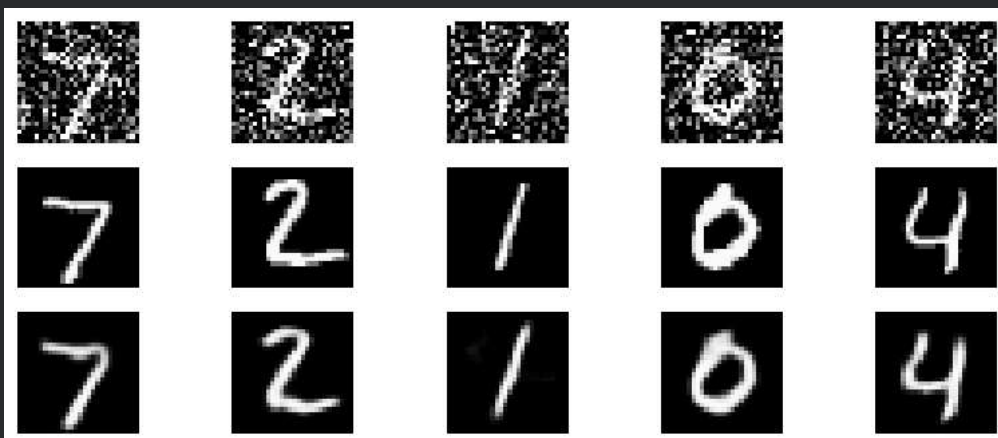
```
1 from keras.metrics import binary_accuracy
2
3 autoencoder.compile(optimizer="adam",
4                     loss="binary_crossentropy",
5                     metrics=[binary_accuracy])
6 print(autoencoder.evaluate(x_test_noisy,x_test))
7 print(binary_accuracy)
```

313/313 ————— 2s 4ms/step - binary_accuracy: 0.0121 - loss: 0.5966
[0.5965953469276428, 0.012066571041941643]
<function binary_accuracy at 0x78c5148e3100>

```
1 decoded_imgs=autoencoder.predict(x_test_noisy)
```

313/313 ————— 2s 3ms/step

```
1 n=5
2 plt.figure(figsize=(10,4))
3 for i in range(n):
4     ax=plt.subplot(3,n,i+1)
5     plt.imshow(x_test_noisy[i].reshape(28,28),cmap="gray")
6     plt.axis("off")
7     ax=plt.subplot(3,n,i+1+n)
8     plt.imshow(x_test[i].reshape(28,28),cmap="gray")
9     plt.axis("off")
10    ax=plt.subplot(3,n,i+1+2*n)
11    plt.imshow(decoded_imgs[i].reshape(28,28),cmap="gray")
12    plt.axis("off")
13 plt.show()
```



cifar10

```
1 import numpy as np
2 import tensorflow as tf
3 import matplotlib.pyplot as plt
4 from tensorflow.keras import layers, models
5 (x_train,_),(x_test,_)=tf.keras.datasets.cifar10.load_data()
6 x_train=x_train.astype("float32")/255.0
7 x_test=x_test.astype("float32")/255.0
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 ————— 13s 0us/step

```
1 noise=1
2 x_train_noisy=x_train+noise*np.random.normal(size=x_train.shape)
3 x_test_noisy=x_test+noise*np.random.normal(size=x_test.shape)
4 x_train_noisy=np.clip(x_train_noisy,0.,1.)
5 x_test_noisy=np.clip(x_test_noisy,0.,1.)
6
7
```

```
1 input_img=layers.Input(shape=(32,32,3))
2 x=layers.Conv2D(32,(3,3),activation="relu",padding="same")(input_img)
```

```

3 x=layers.MaxPooling2D((2,2),padding="same")(x)
4 x=layers.Conv2D(16,(3,3),activation="relu",padding="same")(x)
5 encoder=layers.MaxPooling2D((2,2),padding="same")(x)

```

```

1 x = encoder
2
3 x = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(x)
4 x = layers.UpSampling2D((2, 2))(x)
5
6 x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
7 x = layers.UpSampling2D((2, 2))(x)
8
9 decoder = layers.Conv2D(3, (3, 3), activation='sigmoid', padding='same')(x)

```

```

1 autoencoder=models.Model(input_img,decoder)
2 autoencoder.compile(optimizer="adam",loss="binary_crossentropy")
3 autoencoder.fit(x_train_noisy,x_train,epochs=15,batch_size=128,shuffle=True,validation_data=(x_test_noisy,x_te

```

```

Epoch 1/15
391/391 ————— 12s 18ms/step - loss: 0.6130 - val_loss: 0.6039
Epoch 2/15
391/391 ————— 4s 9ms/step - loss: 0.6020 - val_loss: 0.6015
Epoch 3/15
391/391 ————— 3s 8ms/step - loss: 0.6002 - val_loss: 0.6005
Epoch 4/15
391/391 ————— 3s 8ms/step - loss: 0.5992 - val_loss: 0.5997
Epoch 5/15
391/391 ————— 3s 9ms/step - loss: 0.5985 - val_loss: 0.5990
Epoch 6/15
391/391 ————— 3s 9ms/step - loss: 0.5980 - val_loss: 0.5985
Epoch 7/15
391/391 ————— 3s 8ms/step - loss: 0.5975 - val_loss: 0.5981
Epoch 8/15
391/391 ————— 3s 8ms/step - loss: 0.5971 - val_loss: 0.5987
Epoch 9/15
391/391 ————— 4s 9ms/step - loss: 0.5967 - val_loss: 0.5976
Epoch 10/15
391/391 ————— 3s 8ms/step - loss: 0.5965 - val_loss: 0.5972
Epoch 11/15
391/391 ————— 3s 8ms/step - loss: 0.5963 - val_loss: 0.5971
Epoch 12/15
391/391 ————— 3s 8ms/step - loss: 0.5960 - val_loss: 0.5967
Epoch 13/15
391/391 ————— 4s 9ms/step - loss: 0.5960 - val_loss: 0.5965
Epoch 14/15
391/391 ————— 3s 8ms/step - loss: 0.5958 - val_loss: 0.5964
Epoch 15/15
391/391 ————— 3s 8ms/step - loss: 0.5957 - val_loss: 0.5966
<keras.src.callbacks.history.History at 0x78c4e716f980>

```

▾ binary accuracy of cifar10

```

1 from keras.metrics import binary_accuracy
2
3 autoencoder.compile(optimizer="nadam",
4                     loss="binary_crossentropy",
5                     metrics=[binary_accuracy])
6 print(autoencoder.evaluate(x_test_noisy,x_test))
7 print(binary_accuracy)

```

```

313/313 ————— 3s 4ms/step - binary_accuracy: 0.0121 - loss: 0.5966
[0.5965953469276428, 0.012066571041941643]
<function binary_accuracy at 0x78c5148e3100>

```

```

1 decoded_imgs=autoencoder.predict(x_test_noisy)

```

```

313/313 ————— 2s 5ms/step

```

```

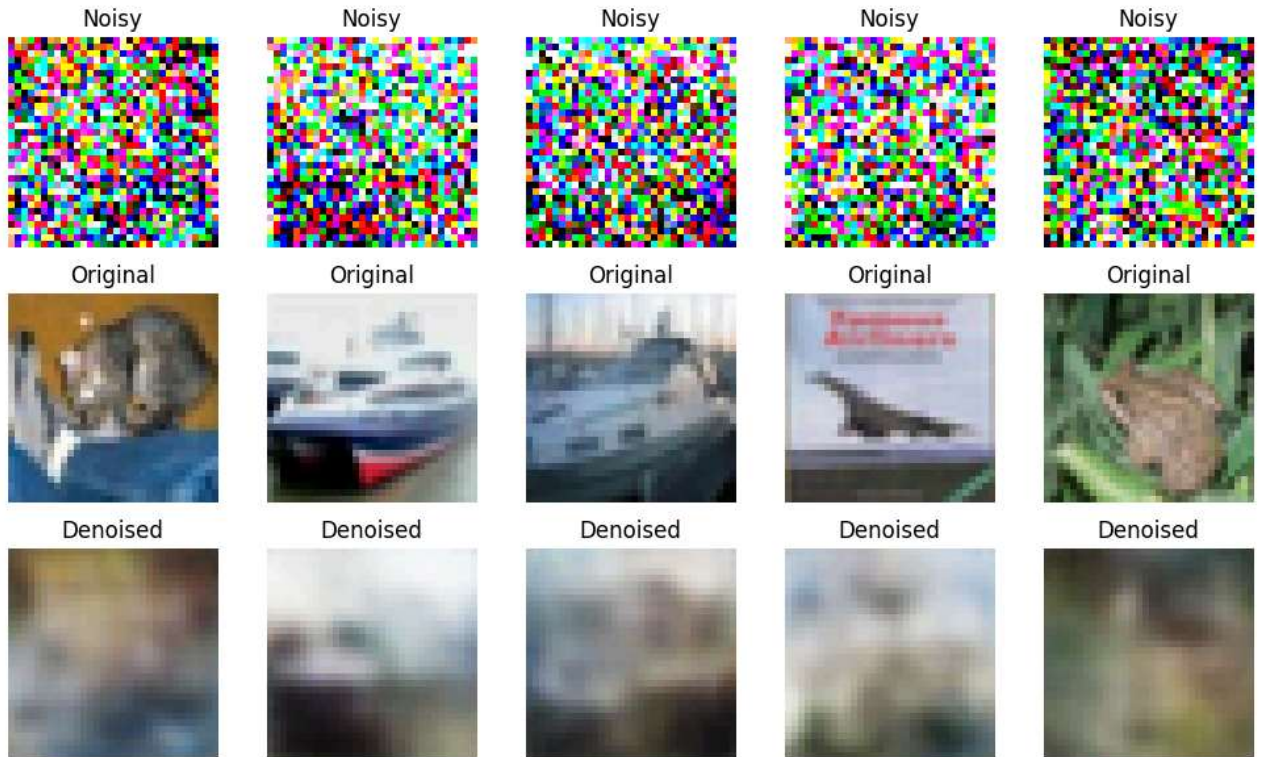
1 n=5
2 plt.figure(figsize=(10,6))
3 for i in range(n):
4
5     ax=plt.subplot(3,n,i+1)
6     plt.imshow(x_test_noisy[i])
7     plt.title("Noisy")
8     plt.axis("off")
9
10
11     ax=plt.subplot(3,n,i+1+n)

```

```

12 plt.imshow(x_test[i])
13 plt.title("Original")
14 plt.axis("off")
15
16
17 ax=plt.subplot(3,n,i+1+2*n)
18 plt.imshow(decoded_imgs[i])
19 plt.title("Denoised")
20 plt.axis("off")
21 plt.tight_layout()
22 plt.show()

```



1 Start coding or [generate](#) with AI.